

Assignment 3

Nischal Pant

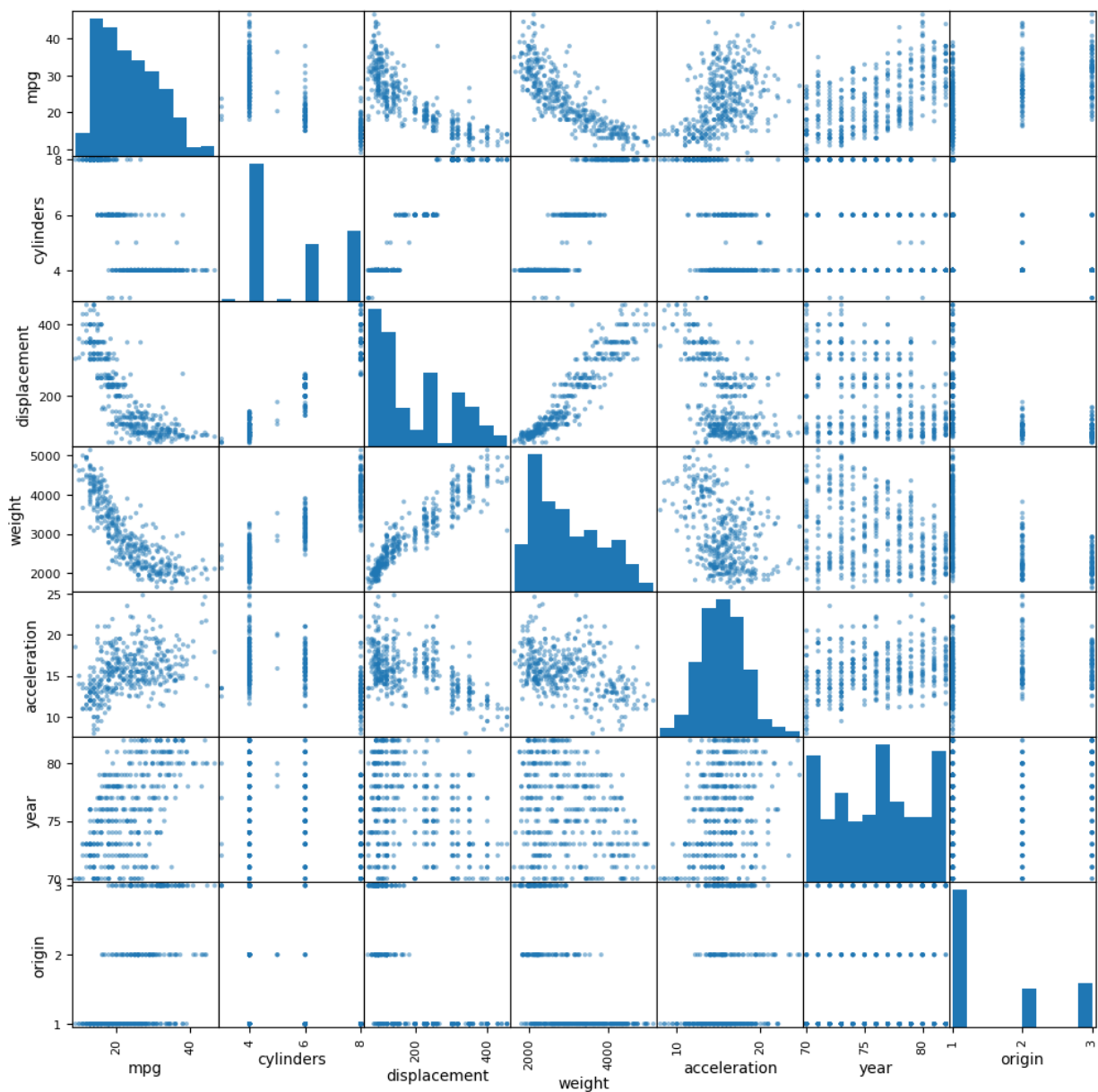
Nov 5th 2023

9. This question involves the use of multiple linear regression on the Auto data set. (a) Produce a scatterplot matrix which includes all of the variables in the data set.

```
In [2]: import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt
```

```
In [3]: auto = pd.read_csv('C:/Users/Nischal/Downloads/auto.csv')

pd.plotting.scatter_matrix(auto, figsize=(12, 12))
plt.show()
```



(b) Compute the matrix of correlations between the variables using the `DataFrame.corr()` method.

```
In [5]: correlation_matrix = auto.corr()
print(correlation_matrix)
```

	mpg	cylinders	displacement	weight	acceleration	\
mpg	1.000000	-0.776260	-0.804443	-0.831739	0.422297	
cylinders	-0.776260	1.000000	0.950920	0.897017	-0.504061	
displacement	-0.804443	0.950920	1.000000	0.933104	-0.544162	
weight	-0.831739	0.897017	0.933104	1.000000	-0.419502	
acceleration	0.422297	-0.504061	-0.544162	-0.419502	1.000000	
year	0.581469	-0.346717	-0.369804	-0.307900	0.282901	
origin	0.563698	-0.564972	-0.610664	-0.581265	0.210084	

	year	origin
mpg	0.581469	0.563698
cylinders	-0.346717	-0.564972
displacement	-0.369804	-0.610664
weight	-0.307900	-0.581265
acceleration	0.282901	0.210084
year	1.000000	0.184314
origin	0.184314	1.000000

```
C:\Users\Nischal\AppData\Local\Temp\ipykernel_11600\905416453.py:1: FutureWarning: The default value of numeric_only in DataFrame.corr is deprecated. In a future version, it will default to False. Select only valid columns or specify the value of numeric_only to silence this warning.
```

```
correlation_matrix = auto.corr()
```

(c) Use the `sm.OLS()` function to perform a multiple linear regression with `mpg` as the response and all other variables except `name` as the predictors. Use the `summarize()` function to print the results. Comment on the output. For instance: i. Is there a relationship between the predictors and the response? Use the `anova_lm()` function from `statsmodels` to answer this question.

```
In [6]: auto = auto.dropna()
y = auto['mpg']
X = auto.drop(columns=['mpg'])
X = sm.add_constant(X)
model = sm.OLS(y, X).fit()
print(model.summary())
```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[6], line 5
      3 X = auto.drop(columns=['mpg'])
      4 X = sm.add_constant(X)
----> 5 model = sm.OLS(y, X).fit()
      6 print(model.summary())

File ~\anaconda3\Lib\site-packages\statsmodels\regression\linear_model.py:906, in OLS.__init__(self, endog, exog, missing, hasconst, **kwargs)
    903     msg = ("Weights are not supported in OLS and will be ignored"
    904           "An exception will be raised in the next version.")
    905     warnings.warn(msg, ValueWarning)
--> 906 super(OLS, self).__init__(endog, exog, missing=missing,
    907                             hasconst=hasconst, **kwargs)
    908 if "weights" in self._init_keys:
    909     self._init_keys.remove("weights")

File ~\anaconda3\Lib\site-packages\statsmodels\regression\linear_model.py:733, in WLS.__init__(self, endog, exog, weights, missing, hasconst, **kwargs)
    731 else:
    732     weights = weights.squeeze()
--> 733 super(WLS, self).__init__(endog, exog, missing=missing,
    734                             weights=weights, hasconst=hasconst, **kwargs)
    735 nobs = self.exog.shape[0]
    736 weights = self.weights

File ~\anaconda3\Lib\site-packages\statsmodels\regression\linear_model.py:190, in RegressionModel.__init__(self, endog, exog, **kwargs)
    189 def __init__(self, endog, exog, **kwargs):
--> 190     super(RegressionModel, self).__init__(endog, exog, **kwargs)
    191     self._data_attr.extend(['pinv_wexog', 'wendog', 'wexog', 'weights'])

File ~\anaconda3\Lib\site-packages\statsmodels\base\model.py:267, in LikelihoodModel.__init__(self, endog, exog, **kwargs)
    266 def __init__(self, endog, exog=None, **kwargs):
--> 267     super().__init__(endog, exog, **kwargs)
    268     self.initialize()

File ~\anaconda3\Lib\site-packages\statsmodels\base\model.py:92, in Model.__init__(self, endog, exog, **kwargs)
     90 missing = kwargs.pop('missing', 'none')
     91 hasconst = kwargs.pop('hasconst', None)
----> 92 self.data = self._handle_data(endog, exog, missing, hasconst,
     93                               **kwargs)
     94 self.k_constant = self.data.k_constant
     95 self.exog = self.data.exog

File ~\anaconda3\Lib\site-packages\statsmodels\base\model.py:132, in Model._handle_data(self, endog, exog, missing, hasconst, **kwargs)
    131 def _handle_data(self, endog, exog, missing, hasconst, **kwargs):
--> 132     data = handle_data(endog, exog, missing, hasconst, **kwargs)
    133     # kwargs arrays could have changed, easier to just attach here
    134     for key in kwargs:

File ~\anaconda3\Lib\site-packages\statsmodels\base\data.py:700, in handle_data(endog, exog, missing, hasconst, **kwargs)
    697     exog = np.asarray(exog)
    699     klass = handle_data_class_factory(endog, exog)
--> 700     return klass(endog, exog=exog, missing=missing, hasconst=hasconst,

```

```

701         **kwargs)

File ~\anaconda3\Lib\site-packages\statsmodels\base\data.py:84, in ModelData.__init__(self, endog, exog, missing, hasconst, **kwargs)
    82     self.orig_endog = endog
    83     self.orig_exog = exog
--> 84     self.endog, self.exog = self._convert_endog_exog(endog, exog)
    86     self.const_idx = None
    87     self.k_constant = 0

File ~\anaconda3\Lib\site-packages\statsmodels\base\data.py:530, in PandasData._convert_endog_exog(self, endog, exog)
    528     else:
    529         exog_dtype = None
--> 530     raise ValueError(
    531         "Pandas data cast to numpy dtype of object. Check input data "
    532         "with np.asarray(data). The types seen were"
    533         f"{endog_dtype} and {exog_dtype}. The data was\n"
    534         f"{orig_endog}\nand\n {orig_exog}\nbefore. After, \n{endog}\n{exog}."
    535     )
    536     return super(PandasData, self)._convert_endog_exog(endog, exog)

ValueError: Pandas data cast to numpy dtype of object. Check input data with np.asarray(data). The types seen wereNone and const
float64
cylinders      int64
displacement   float64
horsepower     object
weight         int64
acceleration   float64
year           int64
origin         int64
name           object
dtype: object. The data was
0      18.0
1      15.0
2      18.0
3      16.0
4      17.0
...
392    27.0
393    44.0
394    32.0
395    28.0
396    31.0
Name: mpg, Length: 397, dtype: float64
and
      const  cylinders  displacement  horsepower  weight  acceleration  year  \
0      1.0         8        307.0         130    3504         12.0    70
1      1.0         8        350.0         165    3693         11.5    70
2      1.0         8        318.0         150    3436         11.0    70
3      1.0         8        304.0         150    3433         12.0    70
4      1.0         8        302.0         140    3449         10.5    70
..      ...         ...         ...         ...         ...         ...    ...
392    1.0         4        140.0          86    2790         15.6    82
393    1.0         4         97.0          52    2130         24.6    82
394    1.0         4        135.0          84    2295         11.6    82
395    1.0         4        120.0          79    2625         18.6    82
396    1.0         4        119.0          82    2720         19.4    82

origin                                name

```

```

0      1  chevrolet chevelle malibu
1      1      buick skylark 320
2      1      plymouth satellite
3      1      amc rebel sst
4      1      ford torino
..      ...
392    1      ford mustang gl
393    2      vw pickup
394    1      dodge rampage
395    1      ford ranger
396    1      chevy s-10

[397 rows x 9 columns]
before. After,
[18.  15.  18.  16.  17.  15.  14.  14.  14.  15.  15.  14.  15.  14.
 24.  22.  18.  21.  27.  26.  25.  24.  25.  26.  21.  10.  10.  11.
   9.  27.  28.  25.  25.  19.  16.  17.  19.  18.  14.  14.  14.  14.
 12.  13.  13.  18.  22.  19.  18.  23.  28.  30.  30.  31.  35.  27.
 26.  24.  25.  23.  20.  21.  13.  14.  15.  14.  17.  11.  13.  12.
 13.  19.  15.  13.  13.  14.  18.  22.  21.  26.  22.  28.  23.  28.
 27.  13.  14.  13.  14.  15.  12.  13.  13.  14.  13.  12.  13.  18.
 16.  18.  18.  23.  26.  11.  12.  13.  12.  18.  20.  21.  22.  18.
 19.  21.  26.  15.  16.  29.  24.  20.  19.  15.  24.  20.  11.  20.
 21.  19.  15.  31.  26.  32.  25.  16.  16.  18.  16.  13.  14.  14.
 14.  29.  26.  26.  31.  32.  28.  24.  26.  24.  26.  31.  19.  18.
 15.  15.  16.  15.  16.  14.  17.  16.  15.  18.  21.  20.  13.  29.
 23.  20.  23.  24.  25.  24.  18.  29.  19.  23.  23.  22.  25.  33.
 28.  25.  25.  26.  27.  17.5 16.  15.5 14.5 22.  22.  24.  22.5 29.
24.5 29.  33.  20.  18.  18.5 17.5 29.5 32.  28.  26.5 20.  13.  19.
 19.  16.5 16.5 13.  13.  13.  31.5 30.  36.  25.5 33.5 17.5 17.  15.5
 15.  17.5 20.5 19.  18.5 16.  15.5 15.5 16.  29.  24.5 26.  25.5 30.5
33.5 30.  30.5 22.  21.5 21.5 43.1 36.1 32.8 39.4 36.1 19.9 19.4 20.2
19.2 20.5 20.2 25.1 20.5 19.4 20.6 20.8 18.6 18.1 19.2 17.7 18.1 17.5
30.  27.5 27.2 30.9 21.1 23.2 23.8 23.9 20.3 17.  21.6 16.2 31.5 29.5
21.5 19.8 22.3 20.2 20.6 17.  17.6 16.5 18.2 16.9 15.5 19.2 18.5 31.9
34.1 35.7 27.4 25.4 23.  27.2 23.9 34.2 34.5 31.8 37.3 28.4 28.8 26.8
33.5 41.5 38.1 32.1 37.2 28.  26.4 24.3 19.1 34.3 29.8 31.3 37.  32.2
46.6 27.9 40.8 44.3 43.4 36.4 30.  44.6 40.9 33.8 29.8 32.7 23.7 35.
23.6 32.4 27.2 26.6 25.8 23.5 30.  39.1 39.  35.1 32.3 37.  37.7 34.1
34.7 34.4 29.9 33.  34.5 33.7 32.4 32.9 31.6 28.1 30.7 25.4 24.2 22.4
26.6 20.2 17.6 28.  27.  34.  31.  29.  27.  24.  36.  37.  31.  38.
36.  36.  36.  34.  38.  32.  38.  25.  38.  26.  22.  32.  36.  27.
27.  44.  32.  28.  31. ]
[[1.0 8 307.0 ... 70 1 'chevrolet chevelle malibu']
 [1.0 8 350.0 ... 70 1 'buick skylark 320']
 [1.0 8 318.0 ... 70 1 'plymouth satellite']
 ...
 [1.0 4 135.0 ... 82 1 'dodge rampage']
 [1.0 4 120.0 ... 82 1 'ford ranger']
 [1.0 4 119.0 ... 82 1 'chevy s-10']].

```

In []: ii. Which predictors appear to have a statistically significant relationship to the response?

In []: iii. What does the coefcient **for** the year variable suggest?

In []: (d) Produce some of diagnostic plots of the linear regression ft **as** described **in** the lab. Comment on any problems you see **with** the ft. Do the residual plots suggest any unusually large outliers?

Does the leverage plot identify any observations **with** unusually high leverage?

In []: (e) Fit some models **with** interactions **as** described **in** the lab. Do any interactions appear to be statistically significant?

In []: (f) Try a few different transformations of the variables, such **as** $\log(X)$,
✓
 X , X^2 . Comment on your findings.

In []: This question should be answered using the Carseats data set.
(a) Fit a multiple regression model to predict Sales using Price, Urban, **and** US.

In []: (b) Provide an interpretation of each coefficient **in** the model. Be careful—some of the variables **in** the model are qualitative!

In []: (c) Write out the model **in** equation form, being careful to handle the qualitative variables properly.

In []: (d) For which of the predictors can you reject the null hypothesis $H_0 : \beta_j = 0$?

In []: (e) On the basis of your response to the previous question, fit a smaller model that only uses the predictors **for** which there **is** evidence of association **with** the outcome.

In []: (f) How well do the models **in** (a) **and** (e) fit the data?

In []: (g) Using the model **from** (e), obtain 95 % confidence intervals **for** the coefficient(s).

In []: (h) Is there evidence of outliers **or** high leverage observations **in** the model **from** (e)?

In []: 14. This problem focuses on the collinearity problem.

(a) Perform the following commands **in** Python:

```
rng = np.random.default_rng(10)
x1 = rng.uniform(0, 1, size=100)
x2 = 0.5 * x1 + rng.normal(size=100) / 10
y = 2 + 2 * x1 + 0.3 * x2 + rng.normal(size=100)
```

The last line corresponds to creating a linear model **in** which y **is** a function of x_1 **and** x_2 . Write out the form of the linear model. What are the regression coefficients?

In [7]: **import** numpy **as** np

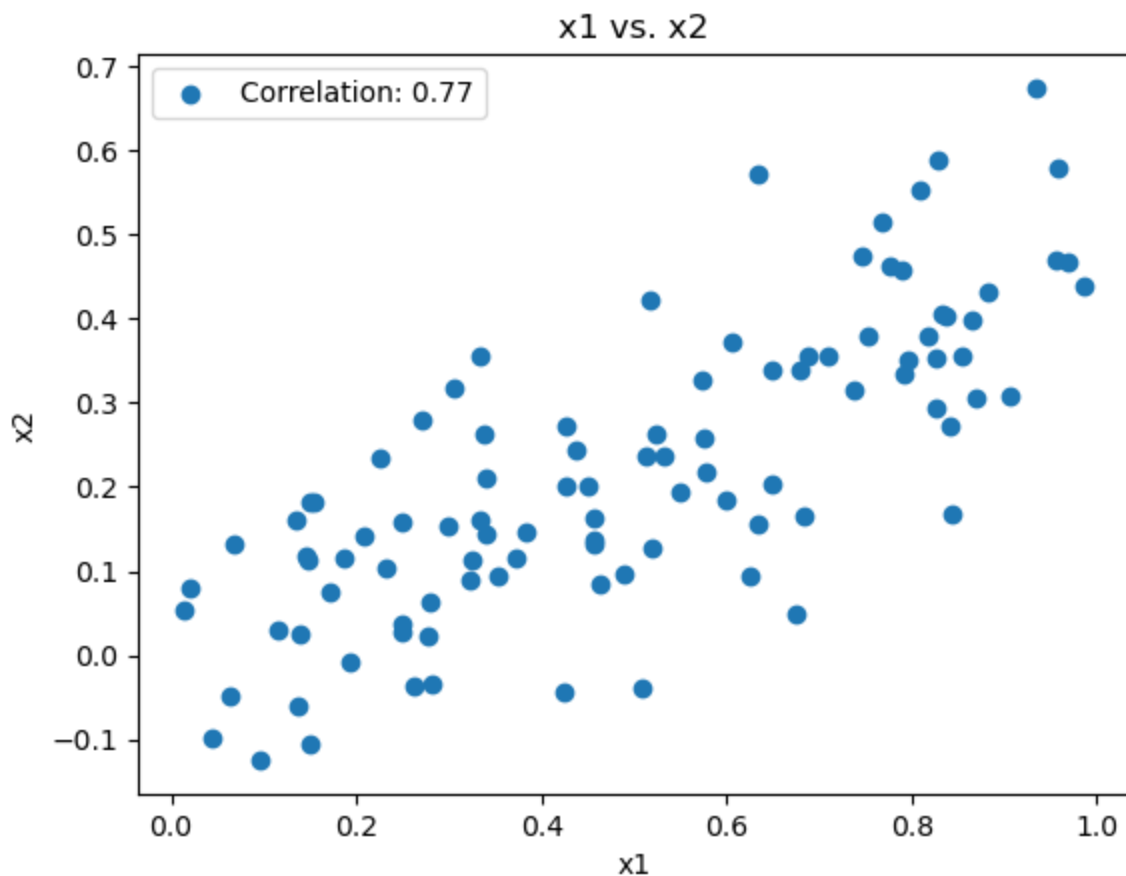
```
rng = np.random.default_rng(10)
x1 = rng.uniform(0, 1, size=100)
x2 = 0.5 * x1 + rng.normal(size=100) / 10
y = 2 + 2 * x1 + 0.3 * x2 + rng.normal(size=100)
```

In []: What are the regression coefficients?

```
 $\beta_0$  (Intercept) = 2
 $\beta_1$  (x1 Coefficient) = 2
 $\beta_2$  (x2 Coefficient) = 0.3
```

In []: (b) What is the correlation between x1 and x2? Create a scatterplot displaying the relationship between the variables.

```
In [9]: correlation = np.corrcoef(x1, x2)[0, 1]
plt.scatter(x1, x2, label=f'Correlation: {correlation:.2f}')
plt.xlabel('x1')
plt.ylabel('x2')
plt.title('x1 vs. x2')
plt.legend()
plt.show()
print(f" x1 and x2: {correlation:.2f}")
```



x1 and x2: 0.77

In []: (c) Using this data, fit a least squares regression to predict y using x1 and x2. Describe the results obtained. What are $\hat{\beta}_0$, $\hat{\beta}_1$, and $\hat{\beta}_2$? How do these relate to the true β_0 , β_1 , and β_2 ? Can you reject the null hypothesis $H_0 : \beta_1 = 0$? How about the null hypothesis $H_0 : \beta_2 = 0$?

In []: (d) Now fit a least squares regression to predict y using only x1. Comment on your results. Can you reject the null hypothesis $H_0 : \beta_1 = 0$?

```
In [10]: X = np.column_stack((x1, x2))
X = sm.add_constant(X)
model = sm.OLS(y, X).fit()
print(model.summary())
```

```

                        OLS Regression Results
=====
Dep. Variable:          y      R-squared:          0.291
Model:                  OLS    Adj. R-squared:      0.276
Method:                 Least Squares    F-statistic:      19.89
Date:                   Sun, 05 Nov 2023    Prob (F-statistic): 5.76e-08
Time:                   21:05:57    Log-Likelihood:    -130.62
No. Observations:       100    AIC:              267.2
Df Residuals:           97    BIC:              275.1
Df Model:               2
Covariance Type:        nonrobust
=====
                        coef      std err          t      P>|t|      [0.025      0.975]
-----
const                1.9579      0.190      10.319      0.000      1.581      2.334
x1                   1.6154      0.527       3.065      0.003      0.569      2.661
x2                   0.9428      0.831       1.134      0.259     -0.707      2.592
=====
Omnibus:              0.051    Durbin-Watson:      1.964
Prob(Omnibus):        0.975    Jarque-Bera (JB):    0.041
Skew:                 -0.036    Prob(JB):            0.979
Kurtosis:             2.931    Cond. No.            11.9
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [ ]: (e) Now fit a least squares regression to predict y using only x2.
Comment on your results. Can you reject the null hypothesis
H0 :  $\beta_1 = 0$ ?
```

```
In [11]: X = x2
X = sm.add_constant(X)
model = sm.OLS(y, X).fit()
print(model.summary())
```

OLS Regression Results

```

=====
Dep. Variable:          y      R-squared:          0.222
Model:                  OLS    Adj. R-squared:       0.214
Method:                 Least Squares  F-statistic:       27.99
Date:                  Sun, 05 Nov 2023  Prob (F-statistic): 7.43e-07
Time:                  21:06:00  Log-Likelihood:    -135.24
No. Observations:      100      AIC:               274.5
Df Residuals:          98       BIC:               279.7
Df Model:               1
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	2.3239	0.154	15.124	0.000	2.019	2.629
x1	2.9103	0.550	5.291	0.000	1.819	4.002

```

=====
Omnibus:                0.191  Durbin-Watson:       1.943
Prob(Omnibus):           0.909  Jarque-Bera (JB):     0.373
Skew:                   -0.034  Prob(JB):             0.830
Kurtosis:                2.709  Cond. No.             6.11
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In []: (f) Do the results obtained in (c)-(e) contradict each other? Explain your answer.

In part (c), a multiple linear regression model was fitted using both x_1 and x_2 as predictor variables to predict y . The results of this model provided coefficients for both x_1 and x_2 , along with their associated p-values. The null hypotheses tested were: $H_0: \beta_1 = 0$ (Null hypothesis for the coefficient of x_1). $H_0: \beta_2 = 0$ (Null hypothesis for the coefficient of x_2). The results of this model would indicate whether both x_1 and x_2 have statistically significant relationships with y . In part (e), a simple linear regression model was fitted using only x_2 as the predictor variable to predict y . The results of this model provided a coefficient for x_2 and its associated p-value. The null hypothesis tested was: $H_0: \beta_1 = 0$ (Null hypothesis for the coefficient of x_2). The results of this model would indicate whether x_2 alone has a statistically significant relationship with y .

(g) Suppose we obtain one additional observation, which was unfortunately mis-measured. We use the function `np.concatenate()` to `np.concaadd` this additional observation to each of x_1 , x_2 and y . `tenate()` $x_1 = \text{np.concatenate}([x_1, [0.1]])$ $x_2 = \text{np.concatenate}([x_2, [0.8]])$ $y = \text{np.concatenate}([y, [6]])$ Re-ft the linear models from (c) to (e) using this new data. What effect does this new observation have on each of the models? In each model, is this observation an outlier? A high-leverage point? Both? Explain your answers.